

1 Context-free grammars

A formal grammar is considered “context free” when its production rules can be applied regardless of the context of a nonterminal. It does not matter which symbols the nonterminal is surrounded by, the single nonterminal on the left hand side can always be replaced by the right hand side.

$$\Sigma = \{0, 1\}, V = \{S\}.$$

Production rules:

$$S \rightarrow 0S1$$

$$S \rightarrow \epsilon$$

Example Derivation (a string over Σ):

$$S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 0011$$

Definition A CFG is a 4-tuple (V, Σ, P, S) where

- V is a finite set of *variables*.
- Σ is a finite set of *terminals*.
- P is a set of *production rules*.
- $S \in V$ is the *start symbol*.

Remark In a CFG, the left-side of production rules are always a **single** variable.

Example $V = \{S\}$, $\Sigma = \{0, 1\}$, $P = \{S \rightarrow 0S1, S \rightarrow \epsilon\}$. P can also be written as $P = \{S \rightarrow 0S1|\epsilon\}$.

Definition The language generated by a grammar G is $L(G) = \{w \mid w \in \Sigma^*, S \xRightarrow{*} w\}$

Example For the previous grammar $\{S \rightarrow 0S1|\epsilon\}$, we have $L(G) = \{0^n 1^n, n \geq 0\}$.

Example $L = \{w \in \{a, b\}^* \mid n_a(w) = n_b(w)\}$

- $\Sigma = \{a, b\}$
- $V = \{S, A, B\}$
- $P :$

$$S \rightarrow aB|bA|\epsilon$$

$$A \rightarrow aS|bAA$$

$$B \rightarrow bS|aBB$$

Example Assume G with $\Sigma = \{0, 1\}$, $V = \{S\}$ and $P : S \rightarrow 0S1|\epsilon$

Claim 1.1 $L(G) = \{0^n 1^n, n \geq 0\}$

Proof

1. If $S \xRightarrow{*} w$, for $w \in \{0, 1\}^*$ then $w = 0^n 1^n$ for some $n \geq 0$.
2. If $w = 0^n 1^n$ for some $n \geq 0$, then $S \xRightarrow{*} w$.

First part: Using proof by induction on the number of steps in the production.

Basis: $S \Rightarrow \epsilon = 0^0 1^0 \checkmark$.

Induction step: Assume that the inductive hypothesis is correct; i.e. if $S \xRightarrow{*} w'$, then $w' = 0^n 1^n$ for all derivations from S with k or fewer steps. Suppose $S \xRightarrow{*} w$ in $k + 1$ steps. The derivation must be:

$$S \rightarrow 0S1 \xRightarrow{*} 0w'1$$

for some w' such that $S \xRightarrow{*} w'$ in k steps. Then, by the inductive hypothesis $w' = 0^n 1^n$ for some n . Hence $w = 0^{n+1} 1^{n+1}$.

Second part: Using proof by induction on the length of w :

Basis: $w = \epsilon$, then $S \Rightarrow \epsilon$.

Induction step: Let $w = 0^n 1^n$, $n \geq 1$. Consider $w = 0w'1$ where $w' = 0^{n-1} 1^{n-1}$. By the inductive hypothesis assume $S \xRightarrow{*} w'$. Then, w can be derived from S :

$$S \rightarrow 0S1 \xRightarrow{*} 0w'1$$

Example Assume G with $\Sigma = \{a, b\}$, $V = \{S, A, B\}$ and P :

$$\begin{aligned} S &\rightarrow aB|bA|\epsilon \\ A &\rightarrow aS|bAA \\ B &\rightarrow bS|aBB \end{aligned}$$

Claim 1.2 $L(G) = \{w \in \{a, b\}^*, n_a(w) = n_b(w)\}$

Proof

1. If $S \xRightarrow{*} w$, then $n_a(w) = n_b(w)$.

2. If $n_a(w) = n_b(w)$, then $S \xRightarrow{*} w$.

First part: Using proof by induction on the number of steps in the production.

Inductive hypothesis:

- If $S \xRightarrow{*} w$, then $n_a(w) = n_b(w)$.
- If $A \xRightarrow{*} w$, then $n_a(w) = n_b(w) + 1$.
- If $B \xRightarrow{*} w$, then $n_a(w) + 1 = n_b(w)$.

Basis:

- $S \rightarrow \epsilon$; $n_a(w) = n_b(w) \checkmark$
- $A \rightarrow aS \rightarrow a$; $n_a(w) = n_b(w) + 1 \checkmark$
- $B \rightarrow bS \rightarrow b$; $n_a(w) + 1 = n_b(w) \checkmark$

Induction step: Assume that the inductive hypothesis is correct for all derivations with n or fewer steps. We will show that it is also true for derivations with $n + 1$ steps.

- Suppose $S \xRightarrow{*} w$ in $n + 1$ steps. Then the derivation begins with either:

$$S \rightarrow aB \xRightarrow{*} aw'$$

or

$$S \rightarrow bA \xRightarrow{*} bw'$$

In the first case, $B \xRightarrow{*} w'$ in n steps, hence, $n_a(w') + 1 = n_b(w')$ by the I.H.

In the second case, $A \xRightarrow{*} w'$ in n steps, hence, $n_a(w') = n_b(w') + 1$ by the I.H.

Either way, $n_a(w) = n_b(w)$.

- Suppose $A \xRightarrow{*} w$ in $n + 1$ steps, $n_a(w) = n_b(w) + 1$. *Exercise.*
- Suppose $B \xRightarrow{*} w$ in $n + 1$ steps, $n_a(w) + 1 = n_b(w)$. *Exercise.*

Second part: Using proof by induction on the length of w .

Inductive hypothesis:

- If $n_a(w) = n_b(w)$, then $S \xRightarrow{*} w$.
- If $n_a(w) = n_b(w) + 1$, then $A \xRightarrow{*} w$.
- If $n_a(w) + 1 = n_b(w)$, then $B \xRightarrow{*} w$.

Basis:

- $w = \epsilon, S \rightarrow \epsilon \checkmark$
- $w = a, A \rightarrow aS \rightarrow a \checkmark$
- $w = b, B \rightarrow bS \rightarrow b \checkmark$

Induction step: Assume that the inductive hypothesis is correct for all strings of length n or shorter. We will show that it is also true for strings of length $n + 1$.

- Suppose $n_a(w) = n_b(w)$. Then w begins with either a or b .

In the first case, $w = aw'$ where $n_a(w') + 1 = n_b(w)$. Hence, by the I.H. we have $B \xRightarrow{*} w'$. Then, we can derive w from S as: $S \rightarrow aB \xRightarrow{*} aw'$.

Similarly, $w = bw'$ where $n_a(w') = n_b(w) + 1$. Hence, by the I.H. we have $A \xRightarrow{*} w'$. Then, we can derive w from S as: $S \rightarrow bA \xRightarrow{*} bw'$.

Either way, $n_a(w) = n_b(w)$.

- Suppose $n_a(w) = n_b(w) + 1$. *Exercise.*
- Suppose $n_a(w) + 1 = n_b(w)$. *Exercise.*

Some more examples:

- $\Sigma = \{ (,) \}$
 $L = \{ w \in \Sigma^* \mid w \text{ is a string of properly nested parentheses} \}$.
 Eg. $()$, $()()$, $((()))$, $\dots$

$$G = (\{S\}, \Sigma, P, S)$$

$$P : S \rightarrow (S) \mid SS \mid \epsilon$$

- $\Sigma = \{ \times, +, a, b, 0, 1 \}$

L : set of arithmetic expressions with variable names over $a, b, 0, 1$. A variable name must begin with an alphabetical character.

$$V = \{E, I\} (\text{Expressions and Identifiers})$$

$$G = (V, \Sigma, P, E)$$

$$P : E \rightarrow E \times E \mid E + E \mid I$$

$$I \rightarrow a \mid b \mid Ia \mid Ib \mid I0 \mid I1$$

1.1 Derivation Trees (parse trees)

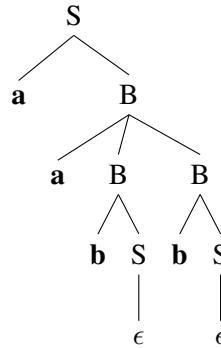
Take:

$$\begin{aligned} S &\rightarrow aB|bA|\epsilon \\ A &\rightarrow aS|bAA \\ B &\rightarrow bS|aBB \end{aligned}$$

Consider $w = aabb$:

$$S \rightarrow aB \rightarrow aaBB \rightarrow aabSB \rightarrow aabB \rightarrow aabb$$

Derivation tree for this derivation:



Ambiguity: informally, you can derive the same string in two different ways.

Example $G = \{\{E\}, \{\times, +, a, b\}, P, E\}$ and $P : E \rightarrow E + E | E \times E | a | b$

Consider $a + b \times a$

You can derive it as:

$$E \rightarrow E + E \rightarrow a + E \rightarrow a + E \times E \rightarrow a + b \times a$$

$$E \rightarrow E \times E \rightarrow E + E \times E \rightarrow a + b \times a$$

But not every different derivation has an ambiguity:

Consider $a + b$. This is not an ambiguity:

$$E \rightarrow E + E \rightarrow a + E \rightarrow a + b$$

$$E \rightarrow E + E \rightarrow E + b \rightarrow a + b$$

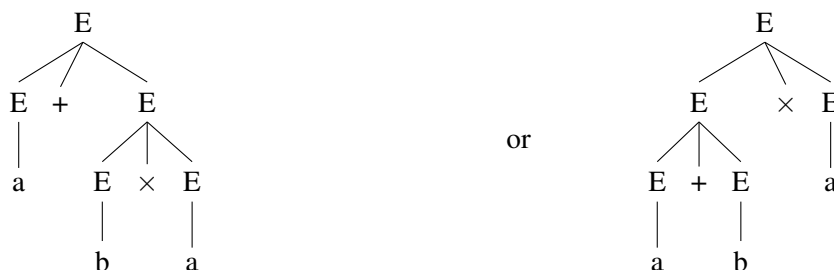
Definition A derivation is “leftmost” if at every step we replace the leftmost variable.

Definition Ambiguity: A string is “ambiguously” in a CFG G if it has two or more different leftmost derivations. Grammar G is “ambiguous” if it generates some string ambiguously.

Example The grammar G above is ambiguous since $a + b \times a$ is derived ambiguously.

Theorem 1.3 A string is ambiguous iff it has two different parse trees.

Example $a + b \times a$:



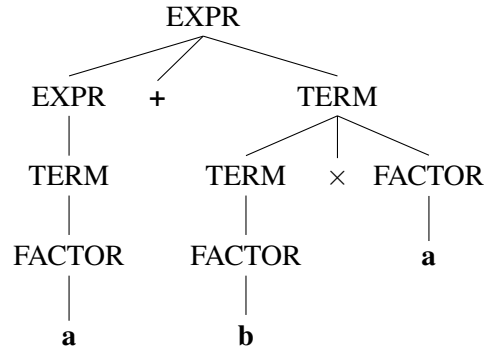
Note that some languages derived by an ambiguous grammar can also be derived by a non-ambiguous grammar as well.

Example $G_2 = \{\{EXPR, TERM, FACTOR\}, \{\times, +, a, b\}, P, EXPR\}$.
 $P :$

$$\begin{aligned} EXPR &\rightarrow EXPR + TERM | TERM \\ TERM &\rightarrow TERM \times TERM | FACTOR \\ FACTOR &\rightarrow a | b \end{aligned}$$

Consider $a + b \times a$

$$EXPR \rightarrow EXPR + TERM \rightarrow TERM + TERM \rightarrow FACTOR + TERM \rightarrow a + TERM \xRightarrow{*} a + b \times a$$



Definition A context-free language (CFL) is called “inherently ambiguous” if it can be produced only by ambiguous grammars.

Example $L = \{a^i b^j c^k | i = j \text{ or } j = k\}$. Consider $w = a^n b^n c^n$ for some $n \geq 0$,

1.2 Regular Grammars

If all productions of a CFG are of the form

$$A \rightarrow wB \quad \text{or}$$

$$A \rightarrow w$$

where $A, B \in V$, $w \in \Sigma^*$, this is called a “right-linear grammar”.

Similarly, if all productions are of the form

$$A \rightarrow Bw \quad \text{or}$$

$$A \rightarrow w$$

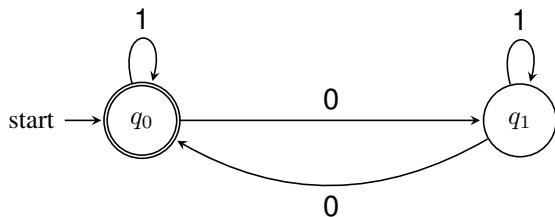
where $A, B \in V$, $w \in \Sigma^*$, this is called a “left-linear grammar”.

If a grammar is left-linear or right-linear, it is called “regular”.

Theorem 1.4 A language is regular iff it can be produced by a regular grammar.

Example $L = \{w \in \{0, 1\}^* | n_0(w) \text{ is even}\}$.

DFA:



RG:

$$S \rightarrow 0A | 1S | \epsilon$$

$$A \rightarrow 0S | 1A$$

1.3 Simplification of a CFG

We can simplify a grammar by eliminating in the following order:

1. ϵ productions - Algorithm 1
2. unit productions - Algorithm 2
3. useless variables - Algorithm 3
4. unreachable symbols - Algorithm 4

1.3.1 Removing ϵ -productions

If $\epsilon \notin L(G)$ we can eliminate all ϵ -productions.

Definition If $A \xRightarrow{*} \epsilon$, then A is “nullable”

Algorithm 1 Remove ϵ -productions

```

if  $A \rightarrow x_1x_2 \dots x_n \in P$  then
    add  $A \rightarrow \alpha_1\alpha_2 \dots \alpha_n$  to  $P'$  where not all  $\alpha_i$ s are  $\epsilon$ :
    if  $x_i$  is not nullable then
         $\alpha_i = x_i$ 
    end if
    if  $x_i$  is nullable then
         $\alpha_i = x_i$  or  $\alpha_i = \epsilon$ 
    end if
end if

```

Example P :

$$S \rightarrow ABaC$$

$$A \rightarrow BC$$

$$B \rightarrow b|\epsilon$$

$$C \rightarrow c|\epsilon$$

Nullable: A, B, C

P' :

$$S \rightarrow ABaC|BaC|AaC|ABa|aC|Ba|Aa|a$$

$$A \rightarrow BC|C|B$$

$$B \rightarrow b$$

$$C \rightarrow c$$

1.3.2 Unit Productions

Productions like $A \rightarrow B$

Algorithm 2 Remove unit productions

```

 $P' =$  all non-unit productions of  $P$ 
if  $A \xRightarrow{*} B$  for  $A, B \in V$  and  $B \rightarrow \alpha$  is a non-unit production then
    add  $A \rightarrow \alpha$  to  $P'$ 
end if

```

Example P :

$$S \rightarrow aBCD$$

$$B \rightarrow bB$$

$$B \rightarrow C$$

$$C \rightarrow D$$

$$D \rightarrow d$$

Note that $B \xRightarrow{*} C$ and C has no non-unit productions. For $C \xRightarrow{*} D$ and $B \xRightarrow{*} D$, $D \rightarrow d$.

Initialization: $P' : S \rightarrow aBCD, B \rightarrow bB, D \rightarrow d$.

Second step: $P' : S \rightarrow aBCD, B \rightarrow bB|d, C \rightarrow d, D \rightarrow d$.

1.3.3 Eliminating useless variables

In a grammar $G = \{V, \Sigma, P, S\}$, a variable X is useful if there is a derivation

$$S \xRightarrow{*} \alpha X \beta \xRightarrow{*} w$$

for some α, β ; and $w \in \Sigma^*$. Otherwise X is “useless”.

Algorithm 3 Remove useless variables

Put every variable A into V' such that $A \rightarrow w$ for some $w \in \Sigma^*$

repeat

if $(B \rightarrow x_1x_2 \dots x_n) \in P$ where each $x_i \in (\Sigma \cup V')$ **then**

 put B into V'

end if

until no new variables are added to V'

Remove all productions that contain useless variables

Example $V = \{S, A, B, C\}, \Sigma = \{a, b, c\}$

P :

$$S \rightarrow AB|aA$$

$$A \rightarrow a$$

$$C \rightarrow ca$$

Initialization:

$$V' = \{A, C\}$$

Loop: S will be added to V' because aA is terminal since we know A will produce terminal string ($A \in V'$).

$$V' = \{A, C, S\}$$

Stop. B is useless. Remove $S \rightarrow AB$ from P .

1.3.4 Remove unreachable symbols

Example $V = \{A, C, S\}$,

P :

$$S \rightarrow aA$$

$$A \rightarrow a$$

$$C \rightarrow ca$$

Initialization: $V' = \{S\}, \Sigma' = \emptyset$

Iteration 1: $V' = \{S, A\}, \Sigma' = \{a\}$

Iteration 2: $\times$

$V' = \{S, A\}, \Sigma' = \{a\}, P' : S \rightarrow aA, A \rightarrow a$.

Algorithm 4 Remove unreachable symbols

Place S in V'
repeat
 if $A \in V'$ and $A \rightarrow \alpha \in P$ **then**
 add all variables in α to V'
 add all terminals in α to Σ'
 end if
until no new variable or terminal is added

Remark To remove useless symbols, first apply Algorithm 3, then Algorithm 4. If you reverse the order, it may not work.

Example $S \rightarrow AB|c, A \rightarrow a$

Alg 3: $S \rightarrow c, A \rightarrow a$

Alg 4: $S \rightarrow c$

If we do it the other way:

Alg 4: $S \rightarrow AB|c, A \rightarrow a$

Alg 3: $S \rightarrow c, A \rightarrow a, \quad \times$

1.4 Chomsky Normal Form (CNF)

If all productions in a CFG are of the form:

$$A \rightarrow BC \quad \text{or}$$

$$A \rightarrow a$$

where $A, B, C \in V$ and $a \in \Sigma$, then that CFG is in *Chomsky Normal Form*.

Theorem 1.5 Any CFL without ϵ can be generated by a grammar in CNF. Hence every CFG can be converted into CNF.

1.4.1 Conversion into CNF

- First, simplify the grammar and obtain $G = (V, \Sigma, P, S)$ without any useless variables/symbols, ϵ -, or unit productions.
- If there is a single symbol on the right, it is a terminal; i.e. no unit productions on the left.
- Consider a production $A \rightarrow X_1 \dots X_m$ ($m \geq 2$).
 - If X_i is a terminal a , introduce variable C_a and production $C_a \rightarrow a$; replace X_i by C_a .
 - For each production $A \rightarrow B_1 \dots B_m, m > 2$, add new variable $D_1, D_2, \dots, D_{m-2}$, and replace this production with:

$$A \rightarrow B_1 D_1$$

$$D_1 \rightarrow B_2 D_2$$

$$D_2 \rightarrow B_3 D_3$$

...

$$D_{m-2} \rightarrow B_{m-1} B_m$$

Example

$$\begin{array}{l}
S \rightarrow aB|bA \\
A \rightarrow aS|bAA|a \\
B \rightarrow bS|aBB|b \\
\hline
S \rightarrow C_a B|C_b A \\
A \rightarrow C_a S|C_b AA|a \\
B \rightarrow C_b S|C_a BB|b \\
C_a \rightarrow a \\
C_b \rightarrow b \\
\hline
S \rightarrow C_a B|C_b A \\
A \rightarrow C_a S|C_b D_1|a \\
B \rightarrow C_b S|C_a D_2|b \\
D_1 \rightarrow AA \\
D_2 \rightarrow BB \\
C_a \rightarrow a \\
C_b \rightarrow b
\end{array}$$

If $\epsilon \in L(G)$ then we allow $S \rightarrow \epsilon$ for the start variable S .

Construction: After converting G into CNF, add a new start variable S_0 and $S_0 \rightarrow S|\epsilon$.

Remark Parsing a grammar in CNF can be achieved using the *CYK Algorithm* (Cocke-Younger-Kasami), which employs *bottom-up parsing* and dynamic programming.

1.5 Greibach Normal Form (GNF)

If all productions in a CFG are of the form:

$$A \rightarrow a\alpha$$

where $\alpha \in V^*$ and $a \in \Sigma$, then that CFG is in *Greibach Normal Form*.

Theorem 1.6 Every CFL can be generated by a grammar in GNF.

Example

$$\begin{array}{l}
S \rightarrow aB|bA \\
A \rightarrow aS|bAA|a \\
B \rightarrow bS|aBB|b
\end{array}$$

Remark Given a grammar in GNF and a derivable string in the grammar with length n , any *top-down parser* will halt at depth n .

1.6 Parsing

Definition Top-down parsing: Look at the highest level of the parse tree and work down using the rewriting rules of the grammar. Easy to construct manually.

- Recursive descent parsers
- Predictive parsers
- Nonrecursive predictive parsers. LL parser: parses from left-to-right; i.e. Leftmost derivation.

Definition Bottom-up parsing: Start from the leaves of the parse tree and work up until you reach the root. They can handle a larger of class of grammars.

- LR parser: rightmost derivation in reverse. Types of LR parser are LALR and SLR parsers. Guaranteed linear run time.

Definition LL(1) grammar: grammars that can be parsed with an LL parser with 1-step lookahead. Most programming languages have LL(1) grammars.

Example Consider grammar for balanced parentheses. $G = \{V, \Sigma, P, B\}$, $V = \{B, R\}$,
 P :

$$B \rightarrow (RB|\epsilon$$

$$R \rightarrow)|(RR$$

$$w = (()())():$$

$$B \rightarrow (RB \rightarrow ((RRB \rightarrow (()RB \rightarrow (()B \rightarrow (())(RB \rightarrow (()())B \rightarrow (()())()$$

1.7 Chomsky hierarchy

Classes of normal grammars:

1. Type-0 grammars, or *unrestricted grammars* include all formal grammars that generate all languages that can be recognized by a Turing machine (*recursively enumerable languages*).
2. Type-1 grammars, or *context-sensitive grammars* generate context-sensitive languages.
3. Type-2 grammars, or *context-free grammars* generate context-free languages.
4. Type-3 grammars, or *regular grammars* generate regular languages.

Hierarchy: Type-3 $\subset$ Type-2 $\subset$ Type-1 $\subset$ Type-0.

Grammar	Languages	Automaton	Production rules
Type-0	Recursively enumerable	Turing machine	$\alpha \rightarrow \beta$ (no restrictions)
Type-1	Context-sensitive	Linear-bounded non-deterministic Turing machine	$\alpha A \beta \rightarrow \alpha \gamma \beta$
Type-2	Context-free	Non-deterministic pushdown automaton	$A \rightarrow \gamma$
Type-3	Regular	Finite state automaton	$A \rightarrow a$ and $A \rightarrow aB$